

Lab 3 Exam

准备工作：创建并切换到 lab3-exam 分支

请在**自动初始化分支后**，在开发机上依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab3-exam
```

初始化的 lab3-exam 分支基于课下完成的 lab3 分支，并且在 tests 目录下添加了 lab3_SRTF 样例测试目录。

题目描述

课下我们在 MOS 系统中实现了时间片轮转算法（Round-Robin, RR）用于进程调度。在本题中，我们将实现抢占式短作业优先（Shortest Remaining Time First, SRTF），用于调度特定任务进程。

题目要求

在本题中，你需要实现函数 env_create_srtf 用于创建 SRTF 调度进程，并返回指向被创建进程的进程控制块的指针。该函数声明如下：

```
struct Env *env_create_srtf(const void *binary, size_t size, int runtime);
```

其中，参数 binary、size 与 env_create 函数中的定义相同，runtime 为 SRTF 调度参数，以时间片为单位：runtime 表示进程总共需要运行的时间片数量。

在本题中，我们将 MOS 系统两次时钟中断之间的间隔定义为一个时间片。

对于每个使用 env_create_srtf 创建的进程，你需要使用 SRTF 算法来进行调度（详见调度规则和示例）。

调度规则

1. 新增的 SRTF 算法与 MOS 原有的 RR 算法拥有**各自独立**的调度队列。SRTF 调度队列中包含所有使用 env_create_srtf 创建的进程，而 RR 调度队列（MOS 原有的调度队列）中包含所有使用 env_create 创建的进程。
2. 当时钟中断产生时，若 SRTF 调度队列中存在尚未运行完所需时间片（runtime）的进程，则选取其中**剩余运行时间最少**的进程调度。如果多个进程的剩余运行时间相同，则选择 env_id **最小**的进程调度。
3. 从 SRTF 调度队列选取进程后，你仅需使其**运行一个时间片**，并在下个时钟中断产生时根据第二条规则，重新选择进程调度。本题中要实现的 SRTF 调度算法不受 yield 参数和进程优先级的影响，只与进程的**剩余运行时间**有关。
4. 如果 SRTF 调度队列为空，或其中的所有进程均已运行完所需的时间片（即剩余运行时间为 0），则使用课下实现的 RR 算法调度 RR 调度队列中的进程。需要注意的是，SRTF 调度的进程可以在**任何时刻**抢占 RR 调度的进程，且 RR 调度的进程运行的时间片在被 SRTF 抢占后**不发生变化**。例如，如果 RR 算法选择调度一个优先级为 5 的进程 A，并已经使其运行了 3 个时间片，此时 SRTF 队列中产生了可以被调度的进程 B，则进程 B 会抢占进程 A 的运行，直至进程 B 及其它 SRTF 进程运行完毕，SRTF 队列为空或所有进程均结束，进程 A 继续运行剩余的 2 个时间片。

示例

以下示例展示：SRTF 进程中途抢占 RR 进程，以及 SRTF 全部结束后恢复 RR 调度。

进程名	类型	优先级	Runtime	创建时机
A	RR	5	-	初始创建
B	RR	1	-	初始创建
C	SRTF	-	2	第 3 个时间片后创建

初始时只有 RR 队列，且当前正在运行 A。

- **第 0-2 个时间片**：SRTF 队列为空，按 RR 调度 A。A 已运行 3 个时间片，还剩 2 个 RR 时间片。
- **第 3 个时间片后**：创建 SRTF 进程 C (runtime=2) 。
- **第 4 个时间片**：到达下一次调度点，C 抢占 A。C 剩余时间 2 -> 1。
- **第 5 个时间片**：继续调度 C。C 剩余时间 1 -> 0，C 运行完毕并移出 SRTF 调度队列。
- **第 6 个时间片及以后**：SRTF 队列再次为空，恢复 RR 调度。A 继续运行其剩余的 2 个时间片（不会重置为 5）；A 用完后按 RR 规则调度 B。

参考实现思路

本参考实现基于 LIST 结构，你也可以采取 TAILQ 结构来实现，满足题目要求即可。

本题参考实现思路如下，你也可以采取其它思路，满足题目要求即可。

1. 在 `include/env.h` 中添加以下声明：

```
LIST_HEAD(Env_srtf_sched_list, Env);

extern struct Env_srtf_sched_list env_srtf_sched_list; // SRTF 调度队列

struct Env *env_create_srtf(const void *binary, size_t size, int runtime);
```

2. 在 `kern/env.c` 中添加 `env_srtf_sched_list` 的定义，并在 `env_init` 函数中初始化 `env_srtf_sched_list`。

```
struct Env_srtf_sched_list env_srtf_sched_list; // SRTF 调度队列
```

3. 在 `include/env.h` 的 `Env` 结构体中添加以下字段：

```
LIST_ENTRY(Env) env_srtf_sched_link; // 构造 env_srtf_sched_list 的链表项
u_int env_total_runtime; // SRTF 调度参数：进程总共需要运行的时间片
u_int env_runtime_left; // 进程剩余需要运行的时间片
```

4. 在 `kern/env.c` 中仿照 `env_create` 实现 `env_create_srtf` 函数，其中需要初始化进程控制块的相关字段，参考如下：

```

struct Env *env_create_srtf(const void *binary, size_t size, int runtime) {
    ...
    // 分配 env 并初始化
    // ...
    // 处理新增的字段
    return e;
}

```

5. 修改 `kern/sched.c` 中的 `schedule` 函数，实现 SRTF 调度算法。

```

void schedule(int yield) {
    /*
    可能的实现思路：
    如果SRTF调度队列中存在尚未运行完所需时间片的进程，则遍历 SRTF 调度队列，选取剩余时间片最少的进程进行调度（如果有多个剩余时间片相同，则选取 env_id 最小的进程）。
    如果找到这样的进程，将其剩余时间片减 1，并调用 `env_run` 调度该进程。
    如果此进程剩余时间片为0，则将其从 SRTF 调度队列中移除。
    如果 SRTF 调度队列为空或所有进程的剩余时间片为 0，则使用 RR 调度算法调度 RR 调度队列中的进程。
    */
    static int count = 0; // remaining time slices of current env
    struct Env *e = curenv; // 请根据提示修改这行代码

    /* 请将 Exercise 3.12 中的代码粘贴至此 */
}

```

提示

1. 你可以使用 `LIST_FOREACH` 宏遍历队列。
2. 为了保证 RR 调度的连续性，当 SRTF 抢占结束后，RR 算法应该从上次被抢占的地方继续运行。可能需要一个静态变量 `struct Env *last_rr_env` 来记录上一次 RR 调度的进程。
`static` 变量在程序运行期间仅在第一次遇到时初始化一次，且函数被多次调用时不会重新初始化该变量，因此适合作为跨调用保存状态（例如记录上一次被 RR 调度的进程）。

评测说明

- 测试程序将调用 `env_create` 和 `env_create_srtf` 创建一批进程。
- `env_free` 不会被测试。
- 评测保证不会出现死锁或空队列情况。
- 在评测中，我们保证进程运行完 `main` 函数后自动进入死循环，而不会主动退出（详见测试目录下的 `entry.S` 文件）。因此，`env_destroy`、`env_free` 函数不会被调用，你无需对它们进行修改。你可以据此结合提示，简化你的实现。
- 不保证进程创建的时机
- 评测中保证不会出现 SRTF 调度队列中无可调度进程且 RR 调度队列为空的情况。
- 对于SRTF调度的进程，可能耗完时间片后程序仍然未结束运行，(例如样例中的 `test_hash3` 和 `test_hash4`)，此时它们会被移出 SRTF 调度队列，不需要额外的处理，你的实现只需保证它们的调度正确即可。

本地测试说明

测试样例如下：

```
#define ENV_CREATE_SRTF(x, y) \  
({ \  
    extern u_char binary_##x##_start[]; \  
    extern u_int binary_##x##_size; \  
    env_create_srtf(binary_##x##_start, (u_int)binary_##x##_size, y); \  
})  
  
void mips_init(u_int argc, char **argv, char **penv, u_int ram_low_size) {  
    printk("init.c:\tmips_init() is called\n");  
  
    mips_detect_memory(ram_low_size);  
    mips_vm_init();  
    page_init();  
    env_init();  
  
    ENV_CREATE_PRIORITY(test_hash1, 1);  
    ENV_CREATE_PRIORITY(test_hash2, 3);  
    ENV_CREATE_SRTF(test_hash3, 5); // runtime = 5  
    ENV_CREATE_SRTF(test_hash4, 2); // runtime = 2  
  
    schedule(0);  
    panic("init.c:\tend of mips_init() reached!");  
}
```

你可以使用：

- `make test lab=3_srtf && make run` 在本地测试上述样例（调试模式）
- `MOS_PROFILE=release make test lab=3_srtf && make run` 在本地测试上述样例（开启优化）

运行若干时间片后，前 7 个时间片应为 SRTF 进程运行（先 2 个 hash4, 后 5 个 hash3）。之后 SRTF 队列为空，转为 RR 调度，按优先级依次运行 hash2 (优先级 3) 和 hash1 (优先级 1)。

```
0: 00002003  
1: 00002003  
2: 00001802  
3: 00001802  
4: 00001802  
5: 00001802  
6: 00001802  
7: 00001001  
8: 00001001  
9: 00001001  
10: 00000800  
11: 00001001  
12: 00001001  
13: 00001001  
14: 00000800  
15: 00001001  
16: 00001001
```

```
env 00001001 reached end pc: 0x00400180, $v0=0x010c4a21
17: 00001001
18: 00000800
19: 00001001
20: 00001001
21: 00001001
22: 00000800
23: 00001001
24: 00001001
25: 00001001
26: 00000800
env 00000800 reached end pc: 0x00400180, $v0=0x00772068
27: 00001001
28: 00001001
29: 00001001
30: 00000800
test finished, halt mos!
```

提交评测 & 评测标准

请在开发机中执行下列命令后，在[课程网站](#)上提交评测。

```
$ cd ~/学号
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

在线评测时，所有的 `.mk` 文件、所有的 `Makefile` 文件、`init/init.c` 以及 `tests/` 和 `tools/` 目录下的所有文件都可能被替换为标准版本，因此请同学们在本地开发时，**不要**在这些文件中编写实际功能所依赖的代码。

具体要求和分数分布如下：

测试点序号	评测说明	分值
1	与样例相同	15
2	混合创建进程测试,仅在初始化时创建进程	20
3	所有进程均使用 <code>env_create_srtf</code> 创建	15
4	不保证进程创建时机	15
5	综合测试	35

测试点依赖关系如下（箭头方向表示依赖方向）：

